Gen AI 시대의 파이썬: 서버 없이 백엔드 서비스 만들기

GitHub 생태계 활용 가이드

이동현

2026년 4월

한국외국어대학교 Social Science & AI융합학부

오늘의 목표

- 별도의 백엔드 서버(인프라) 구축 없이 웹서비스를 만듭니다.

오늘의 목표

- 별도의 백엔드 서버(인프라) 구축 없이 웹서비스를 만듭니다.
- **GitHub 하나로** 백엔드를 대신하는 2가지 패턴을 배웁니다.

오늘의 목표

- 별도의 백엔드 서버(인프라) 구축 없이 웹서비스를 만듭니다.
- **GitHub 하나로** 백엔드를 대신하는 2가지 패턴을 배웁니다.
- GitHub Issues를 데이터베이스로 쓰기 (맛집 보드 만들기)

오늘의 목표

- 별도의 백엔드 서버(인프라) 구축 없이 웹서비스를 만듭니다.
- **GitHub 하나로** 백엔드를 대신하는 2가지 패턴을 배웁니다.
- GitHub Issues를 데이터베이스로 쓰기 (맛집 보드 만들기)
- GitHub Actions로 데이터 자동 수집 (뉴스 대시보드 만들기)

1. GitHub이 곧 서버다

백엔드? 서버?

기존의 한계

우리가 만든 정적 사이트(포트폴리오)는 데이터를 **저장할 곳**이 없었습니다.

서버 구매 → DB 설치 → API 개발
(어렵고 돈이 듭니다!)

백엔드? 서버?

기존의 한계

우리가 만든 정적 사이트(포트폴리오)는 데이터를 **저장할 곳**이 없었습니다.

서버 구매 → DB 설치 → API 개발
(어렵고 돈이 듭니다!)

우리의 새로운 방식

GitHub이 이미 제공하고 있는 강력한 인프라를 서버처럼 활용합니다.

우리의 아키텍처

호스팅 (배포)
GitHub Pages
서빙
프론트엔드
HTML/CSS/JS
fetch()
데이터베이스
Issues API
수집/적재
자동화 서버
Actions

우리의 아키텍처

완전 무료 풀스택

호스팅(Pages) + DB(Issues) + 타이머 서버(Actions) = **비용 0원 ($0)**

2. 실습 A: 맛집 추천 보드 만들기

우리가 만들 서비스 미리보기

우리반 맛집 추천 보드

- 맛집 리스트가 카드 형태로 표시
- 최신 글라스모피즘(Glassmorphism) 뷰
- 데이터는 GitHub Issues에서 가져옴
- 한식, 중식 등 카테고리 필터링

핵심 포인트

아무런 백엔드 없이도,
사용자들이 Issue를 남기면
무한히 늘어나는 동적 데이터를
가지게 됩니다.

GitHub Issues = 데이터베이스?!

Issue 하나 = 데이터 레코드 1개

제목: 맛집 이름 (예: 강남분식)

GitHub Issues = 데이터베이스?!

Issue 하나 = 데이터 레코드 1개

제목: 맛집 이름 (예: 강남분식)

본문: 위치, 가격, 상세내용

GitHub Issues = 데이터베이스?!

Issue 하나 = 데이터 레코드 1개

제목: 맛집 이름 (예: 강남분식)

본문: 위치, 가격, 상세내용

라벨: 필터 및 조건 (`restaurant`, 한식 등)

GitHub Issues = 데이터베이스?!

Issue 하나 = 데이터 레코드 1개

제목: 맛집 이름 (예: 강남분식)

본문: 위치, 가격, 상세내용

라벨: 필터 및 조건 (`restaurant`, 한식 등)

리액션: 좋아요 수 (♡, +1)

API로 실시간 불러오기 (fetch 핵심 코드)

JavaScript에서 아래의 짧은 코드로 데이터를 즉시 가져올 수 있습니다.

app.js 핵심 로직

```
const API_URL = "https://api.github.com/repos/"
              + GITHUB_USERNAME + "/"
              + REPOSITORY_NAME
              + "/issues?labels=restaurant";

const response = await fetch(API_URL);
const issues = await response.json();
// 이제 issues를 화면에 렌더링하면 끝!
```

실습 A: Step-by-Step 가이드

1. **저장소 복사(Fork):** 완성된 맛집 템플릿의 `Use this template` 버튼 클릭

실습 A: Step-by-Step 가이드

1. **저장소 복사(Fork):** 완성된 맛집 템플릿의 `Use this template` 버튼 클릭
2. **Pages 배포 활성화:** `Settings -> Pages`에서 `main` 브랜치 배포

실습 A: Step-by-Step 가이드

1. **저장소 복사(Fork):** 완성된 맛집 템플릿의 `Use this template` 버튼 클릭
2. **Pages 배포 활성화:** `Settings -> Pages`에서 `main` 브랜치 배포
3. **코드 수정:** `app.js`의 `GITHUB_USERNAME` 변수를 내 아이디로 변경 (커밋 푸시)

실습 A: Step-by-Step 가이드

1. **저장소 복사(Fork):** 완성된 맛집 템플릿의 `Use this template` 버튼 클릭
2. **Pages 배포 활성화:** `Settings -> Pages`에서 `main` 브랜치 배포
3. **코드 수정:** `app.js`의 `GITHUB_USERNAME` 변수를 내 아이디로 변경 (커밋 푸시)
4. **데이터 등록:** 내 레포의 `Issues` 탭에서 '`restaurant`' 라벨 달고 맛집 생성

실습 A: Step-by-Step 가이드

1. **저장소 복사(Fork):** 완성된 맛집 템플릿의 `Use this template` 버튼 클릭
2. **Pages 배포 활성화:** `Settings -> Pages`에서 `main` 브랜치 배포
3. **코드 수정:** `app.js`의 `GITHUB_USERNAME` 변수를 내 아이디로 변경 (커밋 푸시)
4. **데이터 등록:** 내 레포의 `Issues` 탭에서 '`restaurant`' 라벨 달고 맛집 생성
5. **동작 확인:** GitHub Pages 접속 후 연동 및 시각 효과 확인!

3. 실습 B: 뉴스 브리핑 자동 수집 봇

우리가 만들 서비스 미리보기

AI 최신 뉴스 대시보드

- 로봇이 매일 아침 뉴스를 배달
- 대시보드에서 최신 뉴스 열람
- 수집 출처별 차트(도넛) 시각화 제공

핵심 포인트

파이썬 수집 스크립트가 내 컴퓨터가 아닌, **GitHub 서버** 위에서 매일 정해진 시각에 돕니다!

GitHub Actions = 로봇 비서

기존의 한계

내 컴퓨터 전원을 끄면 작성해둔 스크립트 실행도 멈춥니다.

GitHub Actions = 로봇 비서

기존의 한계

내 컴퓨터 전원을 끄면 작성해둔 스크립트 실행도 멈춥니다.

GitHub Actions

정해진 시간에 나를 대신해 코드를 실행해주는 **무료 클라우드 서버**입니다.

GitHub Actions = 로봇 비서

기존의 한계

내 컴퓨터 전원을 끄면 작성해둔 스크립트 실행도 멈춥니다.

GitHub Actions

정해진 시간에 나를 대신해 코드를 실행해주는 **무료 클라우드 서버**입니다.

YAML 파일(.yml) 단 1개로 이 로봇(Runner)에게 작업 지시를 내립니다.

지시서(YAML) 작성법

.github/workflows/collect.yml

```yaml
name: News Collector

on:
  schedule:
    - cron: '0 0 * * *'        # Run automated
  workflow_dispatch:           # Run manual switch

jobs:
  collect:
    permissions:
      contents: write          # Push auth needed
    runs-on: ubuntu-latest
    steps:
```

실습 B: Step-by-Step 가이드

1. **저장소 복사(Fork):** 공유된 뉴스 대시보드 템플릿 생성

실습 B: Step-by-Step 가이드

1. **저장소 복사(Fork):** 공유된 뉴스 대시보드 템플릿 생성
2. **Pages 배포 활성화:** `Settings -> Pages` 설정

실습 B: Step-by-Step 가이드

1. **저장소 복사(Fork):** 공유된 뉴스 대시보드 템플릿 생성
2. **Pages 배포 활성화:** `Settings -> Pages` 설정
3. **수동 수집 테스트:** `Actions` 탭 진입 → `Run workflow` 수동 실행

실습 B: Step-by-Step 가이드

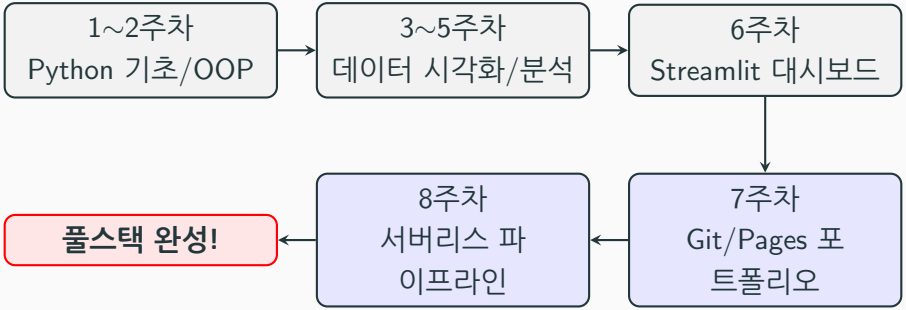
1. **저장소 복사(Fork):** 공유된 뉴스 대시보드 템플릿 생성
2. **Pages 배포 활성화:** `Settings -> Pages` 설정
3. **수동 수집 테스트:** `Actions` 탭 진입 → `Run workflow` 수동 실행
4. **로그 확인:** 초록색 체크마크 확인 시 `data/news.json` 갱신 완료

실습 B: Step-by-Step 가이드

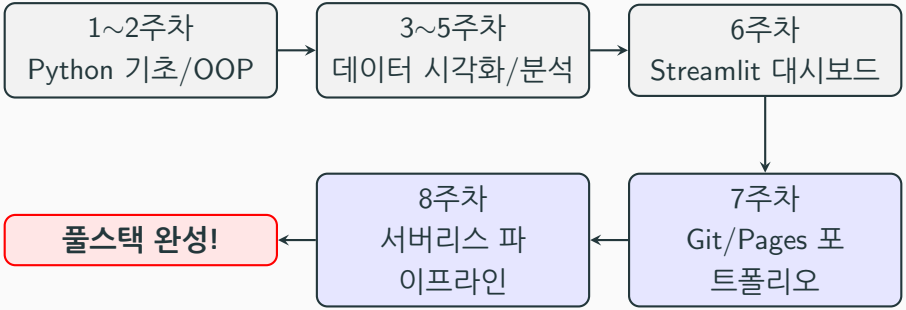
1. **저장소 복사(Fork):** 공유된 뉴스 대시보드 템플릿 생성
2. **Pages 배포 활성화:** `Settings -> Pages` 설정
3. **수동 수집 테스트:** `Actions` 탭 진입 → `Run workflow` 수동 실행
4. **로그 확인:** 초록색 체크마크 확인 시 `data/news.json` 갱신 완료
5. **결과 확인:** GitHub Pages로 접속해 뉴스 및 차트 로딩 확인

4. 8주 과정 Wrap-Up

우리의 8주 여정


1~2주차
Python 기초/OOP
3~5주차
데이터 시각화/분석
6주차
Streamlit 대시보드
7주차
Git/Pages 포
트폴리오
8주차
서버리스 파
이프라인
풀스택 완성!

우리의 8주 여정



여러분은 이제 **데이터를 다루고 분석**하는 것부터 **웹서비스 배포와 자동화**까지 혼자서 해낼 수 있습니다!

즐거운 코딩 생활이 되기를!

수고 많으셨습니다.